Low-Level Design (LLD) Document

Project: RAG-Based Customer Support Assistant

1. Module-Level Design

- **Document Processing Module:** Utilizes libraries like PyPDFLoader to extract raw strings from PDF files.
- **Chunking Module:** Utilizes RecursiveCharacterTextSplitter to divide text based on paragraphs and sentences rather than cutting off mid-word.
- **Embedding Module:** A wrapper class that initializes the chosen embedding model.
- **Vector Storage Module:** Manages the connection to ChromaDB, handling the insertion of documents and persistence of the database directory.
- **Retrieval Module:** Takes a string input, embeds it, performs a cosine similarity search in ChromaDB, and returns a list of matched document objects.
- **Query Processing Module:** Formats the user's prompt and manages the chat loop.
- **Graph Execution Module:** Initializes a StateGraph, defines the nodes (functions), adds conditional edges, and compiles the application.
- **HITL Module:** A terminal or UI prompt that halts the graph execution, displays the query to the agent, and captures their input.

2. Data Structures

- **Document Representation:** An object containing page_content (string) and metadata (dictionary with source and page number).
- **Chunk Format:** Same as document representation, but representing a smaller subset of text.
- **Embedding Structure:** A list/array of floating-point numbers.
- **Query-Response Schema:** JSON/Dictionary format containing {"user_query": string, "assistant_response": string}.
- **State Object for Graph (TypedDict):**

Python

```
class GraphState(TypedDict):
    question: str
    context: str
    answer: str
    intent: str
    needs_human: bool
```

3. Workflow Design (LangGraph)

- **Nodes:**

    - retrieve_node: Fetches context from the database and updates the context state.
    - evaluate_node: Prompts the LLM to output "yes" or "no" regarding its confidence in answering the query based on the context. Updates intent state.
    - generate_node: Formats the final answer using the context. Updates answer state.
    - hitl_node: Prompts the human agent. Updates answer state.
- **Edges:**
    - START -> retrieve_node -> evaluate_node
    - evaluate_node -> CONDITIONAL ROUTING
- **State:** The GraphState dictionary is passed sequentially to each node, accumulating data.

4. Conditional Routing Logic

- **Answer Generation Criteria:** If the evaluate_node returns a positive confirmation (e.g., "yes") that the retrieved context contains the factual answer to the user's question, the router directs the flow to the generate_node.
- **Escalation Criteria:**
    - **Low Confidence/Missing Context:** If the context is empty, or the LLM determines the retrieved text does not contain the answer (returns "no").
    - **Complex Query:** If the LLM identifies the query requires reasoning outside the scope of the provided policy document.

5. HITL Design

- **When Escalation is Triggered:** Activated immediately by the conditional router when intent requires escalation.
- **What Happens:** The automated process suspends. A system alert is generated: *"System Alert: AI confidence low. Manual intervention required."* The original user query is displayed to the agent.
- **Integration:** The agent types a response. This string is appended to the answer key in the GraphState with a prefix (e.g., [Agent Assisted]:) and the graph proceeds to the END node.

6. API / Interface Design

- **Input Format:** Plain text string from the user.
- **Output Format:** Plain text string returned to the user interface.
- **Interaction Flow:** A continuous loop (while True) that accepts input, invokes the LangGraph app with the initial state, and prints the result until the user terminates the session.

7. Error Handling

- **Missing Data:** If no PDF is found in the directory on startup, raise a FileNotFoundError with instructions.

- **No Relevant Chunks Found:** If the retriever returns an empty list, automatically set the needs_human flag to True to bypass the LLM evaluation and go straight to HITL.
- **LLM Failure:** Wrap LLM API calls in a try-except block to catch rate limits or connection timeouts, triggering a graceful fallback message asking the user to try again.